

Mantenimiento Predictivo de Turbofanes NASA FD002:

Clasificación Binaria de Estado Crítico mediante Random Forest y Regresión Logística

Martin Jesús Bonilla Sarmiento, Natalia Watson Guevara, Lucia Jimena Cartagena Miranda
Universidad de Ingeniería y Tecnología (UTEC)

(Proyecto 1 – Clasificación)

Abstract—Se presenta un sistema de alerta temprana para la detección de turbofanes en estado crítico ($RUL \leq 30$ ciclos) usando el conjunto de datos NASA FD002. El preprocesamiento incluyó eliminación de sensores de baja varianza, desconfoundización respecto a condiciones operacionales y selección por correlación. Se entrenaron y compararon tres modelos: Random Forest sobre todas las características (RF-All, sin desconfoundización), Random Forest sobre sensores supervivientes (RF-Sel, con desconfoundización) y Regresión Logística sobre sensores supervivientes (LR-Sel). El mejor modelo fue RF-All, logrando un F1-score de 0.8598 con umbral óptimo $\tau^* = 0.661$, detectando el 87.3 % de los motores críticos con 97.1 % de especificidad en la clase sana. El análisis de costo-beneficio muestra un ahorro potencial de \$678.6M respecto al escenario sin detección.

Index Terms—Mantenimiento predictivo, clasificación binaria, Random Forest, regresión logística, NASA CMAPSS FD002, RUL, desconfoundización, F1-score.

I. INGENIERÍA DE DATOS Y FORMULACIÓN

EL dataset NASA FD002 registra series temporales de 260 motores de turbofán bajo **seis condiciones operacionales** distintas (combinaciones de altitud, número de Mach y punto de regulación) y 21 sensores de temperatura, presión y velocidad. Cada fila corresponde a un ciclo de operación de un motor; el registro comienza en el ciclo 1 (motor nuevo) y termina en el ciclo donde ocurre la falla.

A. Definición de la variable objetivo

La variable de interés es la *Vida Útil Restante* (RUL):

$$RUL_i = \text{ciclo_max}(\text{motor}_i) - \text{ciclo_actual}_i \quad (1)$$

A partir del RUL se definió la etiqueta binaria de estado crítico con el estándar de la literatura de alerta temprana de 30 ciclos:

$$y_i = 1 [RUL_i \leq 30] \quad (2)$$

donde $y = 1$ denota estado *Crítico* (requiere intervención inmediata) e $y = 0$ denota estado *Sano*.

B. Partición de datos

Los 260 motores se dividieron al 80/20 **por motor** —no por fila— para evitar filtración temporal entre entrenamiento y validación. Si un motor con 200 ciclos fuera dividido por

TABLE I
VARIABLES DERIVADAS EN LA ETAPA DE PREPROCESAMIENTO

Variable	Descripción
max_cycle	Máximo ciclo observado por motor; obtenido con <code>groupby().max()</code> .
RUL	Vida útil restante: diferencia entre max_cycle y el ciclo actual.
label	Etiqueta binaria $y = 1 [RUL \leq 30]$.
sensor*_deconf	Residuo del sensor tras restar la predicción lineal basada en las tres variables operacionales (<code>op_setting_1/2/3</code>). Elimina el efecto de régimen de vuelo.

filas, el modelo vería ciclos tardíos del mismo motor durante el entrenamiento y ciclos tempranos en validación, lo que constituye data leakage temporal.

- **Entrenamiento:** 208 motores (43 464 filas)
- **Validación:** 52 motores (10 295 filas)

La distribución de clases en ambas particiones es consistente (85 % sano / 15 % crítico en entrenamiento; 84 % / 16 % en validación), reflejando el desbalance natural del problema.

C. Nuevas variables creadas

La Tabla I resume las variables derivadas durante la ingeniería de datos.

D. Preprocesamiento

El pipeline de preprocesamiento constó de tres etapas secuenciales aplicadas exclusivamente sobre el conjunto de entrenamiento para evitar leakage.

(1) **Eliminación por baja varianza.** Se calculó la varianza de cada sensor sobre el conjunto de entrenamiento. Los sensores con varianza inferior al umbral $\tau_v = 0.1$ fueron descartados por aportar información prácticamente nula —un sensor casi constante no puede discriminar entre estados. La Fig. 1 muestra la distribución de varianzas; los sensores eliminados fueron `sensor_16` ($\text{var} = 2.2 \times 10^{-5}$) y `sensor_10` ($\text{var} = 0.016$).

(2) **Desconfoundización operacional.** El dataset FD002 opera bajo seis regímenes de vuelo distintos, lo que introduce una correlación espuria entre sensores y etiqueta mediada por las condiciones operacionales. Antes de la desconfoundización, `sensor_15` presentaba $|\rho| = 0.03$ con la etiqueta; tras ella, $|\rho| = 0.53$, cerca del máximo teórico para este desbalance de clases. Para cada sensor superviviente se ajustó una regresión lineal $\hat{s} = \mathbf{w}^\top \mathbf{o}$ (donde $\mathbf{o}$ son las tres condiciones operacionales) sobre el conjunto de entrenamiento, y el residuo $s - \hat{s}$ fue usado como característica desconfoundizada. Los parámetros se estiman en entrenamiento y se aplican sin reajuste en validación.

(3) **Selección por correlación.** Sobre los sensores desconfoundizados se calculó la correlación de Pearson con la etiqueta binaria (correlación punto-biserial). Aquellos con $|\rho| < 0.03$ fueron eliminados por no ofrecer poder discriminativo medible. Sobrevivieron **11 sensores** de los 19 restantes (véase Apéndice B).

Manejo del desbalance de clases. Dado que la clase crítica representa el 15% de los datos, todos los modelos se entrenaron con `class_weight='balanced'`, que pondera inversamente la frecuencia de cada clase en la función de pérdida, forzando al modelo a no ignorar la clase minoritaria.

II. METODOLOGÍA Y OPTIMIZACIÓN

A. Modelos implementados

Se evaluaron tres configuraciones:

- 1) **RF-All:** Random Forest con las **22 características** (3 variables operacionales + 19 sensores tras filtro de varianza), sin desconfoundización. Evalúa la hipótesis del TA: el RF gestiona las condiciones operacionales de forma nativa.
- 2) **RF-Sel:** Random Forest con las **14 características** supervivientes (3 operacionales + 11 sensores desconfoundizados y filtrados por correlación).
- 3) **LR-Sel:** Regresión Logística con las mismas 14 características, precedida de `StandardScaler` dentro de un `Pipeline` para garantizar que el escalado no cause leakage.

La elección de **Random Forest** se justifica por su capacidad para capturar interacciones no lineales entre sensores, su robustez frente al ruido y su resistencia intrínseca al sobreajuste mediante bagging (cada árbol ve una submuestra aleatoria de datos y características). La **Regresión Logística**, aunque asume frontera de decisión lineal, sirve como línea base interpretable y permite verificar si la relación sensor-fallo puede aproximarse linealmente tras la desconfoundización.

B. Optimización del umbral de decisión

En lugar del umbral por defecto $\tau = 0.5$, se optó por maximizar el F1-score sobre la curva Precision-Recall del conjunto de validación:

$$\tau^* = \arg \max_{\tau} \frac{2 \cdot P(\tau) \cdot R(\tau)}{P(\tau) + R(\tau) + \varepsilon} \quad (3)$$

Este enfoque equilibra falsas alarmas (baja precisión) con motores críticos no detectados (bajo recall), priorizando la utilidad operacional sobre la simple maximización de accuracy.

TABLE II
HIPERPARÁMETROS DE LOS MODELOS ENTRENADOS

Modelo	Parámetro	Valor
RF-All RF-Sel	<code>n_estimators</code>	100
	<code>max_depth</code>	10
	<code>min_samples_leaf</code>	20
	<code>class_weight</code>	balanced
	<code>random_state</code>	42
LR-Sel	<code>C</code>	1.0
	<code>max_iter</code>	1000
	<code>class_weight</code>	balanced

C. Hiperparámetros y justificación

La Tabla II resume los hiperparámetros utilizados. Para el preprocesamiento de **LR-Sel**, se aplicó `StandardScaler` dentro de un `Pipeline` de *scikit-learn*. El `StandardScaler` aplica normalización Z-score ($x' = (x - \mu)/\sigma$) a cada sensor, necesaria porque sensores como `sensor_9` operan en miles mientras `sensor_15` opera en unidades; sin esta normalización, el gradiente quedaría dominado por las magnitudes mayores e impediría la convergencia. El `Pipeline` garantiza que los parámetros de escalado se estimen exclusivamente sobre el conjunto de entrenamiento y se apliquen sin reajuste al de validación, evitando *data leakage*.

Para **Random Forest**, `n_estimators=100` ofrece un balance entre estabilidad del ensamble y tiempo de cómputo. `max_depth=10` limita la profundidad de cada árbol para evitar que memorice patrones específicos de los 208 motores de entrenamiento. `min_samples_leaf=20` exige que cada hoja represente al menos 20 ciclos, evitando divisiones sobre observaciones aisladas. `class_weight=balanced` compensa el desbalance 85%/15% aumentando la penalización por errores en la clase Crítico, y `random_state=42` garantiza reproducibilidad.

Para **LR-Sel**, `C=1.0` controla la regularización L2 ($\lambda = 1/C$): valores menores simplifican el modelo en exceso (*underfitting*), mientras que valores mayores permiten pesos demasiado grandes y sobreajuste. `C=1.0` representa el punto de partida estándar que mantiene el equilibrio sesgo-varianza adecuado para este dataset. `max_iter=1000` asegura que el optimizador converja completamente con los datos escalados.

III. RESULTADOS

A. Análisis de varianza de sensores

La Fig. 1 evidencia una disparidad de cinco órdenes de magnitud entre sensores. Dos sensores (barras rojas) presentan varianza inferior al umbral y fueron descartados por no aportar información discriminativa. El sensor con mayor varianza es `sensor_9` ($\approx 115\,000$), seguido de `sensor_7` y `sensor_18` ($\approx 22\,000$), lo que sugiere que miden fenómenos de alta dinámica térmica o de presión.

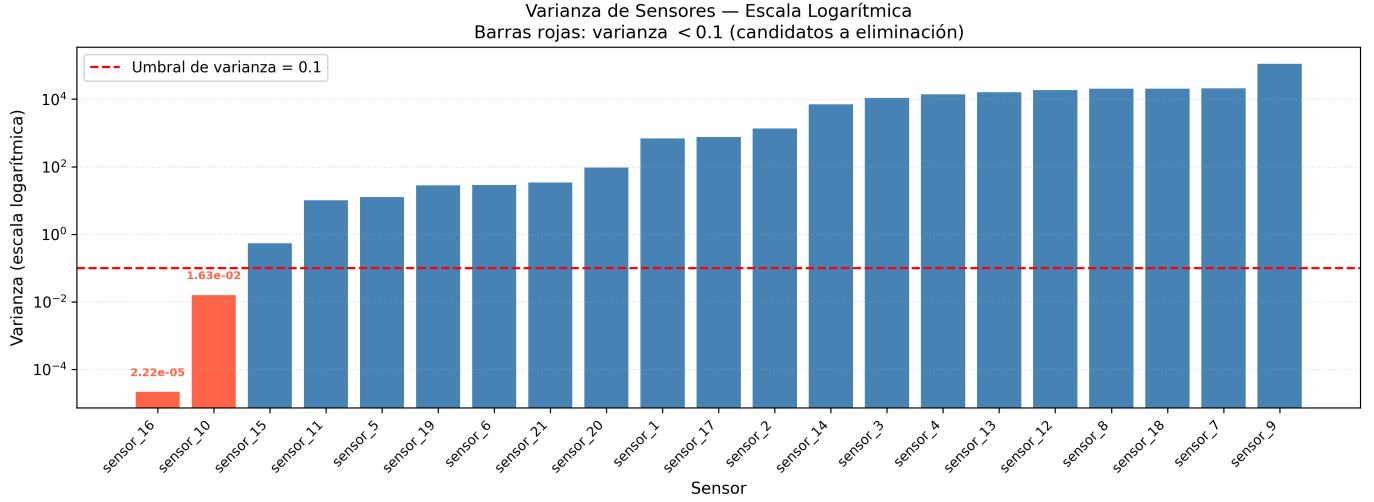


Fig. 1. Varianza de los 21 sensores ordenada de menor a mayor. Las barras rojas (varianza < 0.1) son candidatos a eliminación. La línea roja discontinua indica el umbral $\tau_v = 0.1$.

B. Curvas Precision-Recall

La Fig. 2 muestra las curvas Precision-Recall para los tres modelos. **RF-All** domina sobre RF-Sel en la mayor parte de la curva, y ambos Random Forests superan a LR-Sel, especialmente en la región de alto recall (> 0.7). Todos los modelos superan ampliamente la línea base aleatoria de precisión=0.16 (proporcional a la clase crítica). Las estrellas marcan el punto óptimo de F1 para cada modelo.

C. Matrices de confusión

La Fig. 3 presenta las matrices de confusión normalizadas por fila (porcentaje sobre clase real). La Tabla III resume las métricas cuantitativas.

TABLE III
COMPARACIÓN DE MODELOS EN EL CONJUNTO DE VALIDACIÓN

Modelo	F1	Acc.	Recall	Prec.	τ^*
RF-All	0.8598	0.9554	87.3 %	84.7 %	0.661
RF-Sel	0.8562	0.9540	87.5 %	83.8 %	0.652
LR-Sel	0.8310	0.9459	84.9 %	81.3 %	0.714

Los principales hallazgos son:

- **RF-All** logra el mayor F1 (0.8598) y la mayor especificidad de clase sana (97.1 %), confirmando que el Random Forest gestiona las condiciones operacionales de forma nativa sin necesitar desconfundización previa.
- **RF-Sel** obtiene recall ligeramente superior (87.5 % vs. 87.3 %) y menor costo esperado, pero su F1 global es marginalmente inferior, posiblemente porque la desconfundización lineal elimina parte de la señal útil contenida en interacciones no lineales sensor-condición operacional.
- **LR-Sel** presenta la precisión más baja (81.3 %), generando más falsas alarmas: 314 FP vs. 254 de RF-All, lo que refleja la incapacidad del modelo lineal

para capturar la frontera de decisión no lineal de este problema.

IV. ANÁLISIS DE COSTO-BENEFICIO

A. Definición de la matriz de costos monetaria

Se definen dos escenarios de error con consecuencias económicas asimétricas: un **Falso Negativo** (FN) ocurre cuando el modelo clasifica como Sano un motor en estado Crítico; al no intervenir, el motor falla catastróficamente con costo estimado en \$500 000 (reemplazo del motor más costos operacionales y de seguridad). Un **Falso Positivo** (FP) ocurre cuando el modelo alerta sobre un motor Sano; se realiza un mantenimiento preventivo innecesario con costo estimado en \$15 000. Un **Verdadero Positivo** (VP) representa la detección correcta de un motor Crítico; el costo de mantenimiento programado es idéntico al FP (\$15 000), pero evita la falla catastrófica. Un **Verdadero Negativo** no genera costo adicional.

TABLE IV
MATRIZ DE COSTOS MONETARIA (USD POR EVENTO)

	Predicho: Sano	Predicho: Crítico
Real: Sano	\$0 (TN)	\$15 000 (FP)
Real: Crítico	\$500 000 (FN)	\$15 000 (VP)

La asimetría es de 33:1 entre FN y FP. Esto implica que incluso si el modelo genera varias falsas alarmas, el costo acumulado sigue siendo muy inferior al de dejar pasar un solo motor en falla inminente.

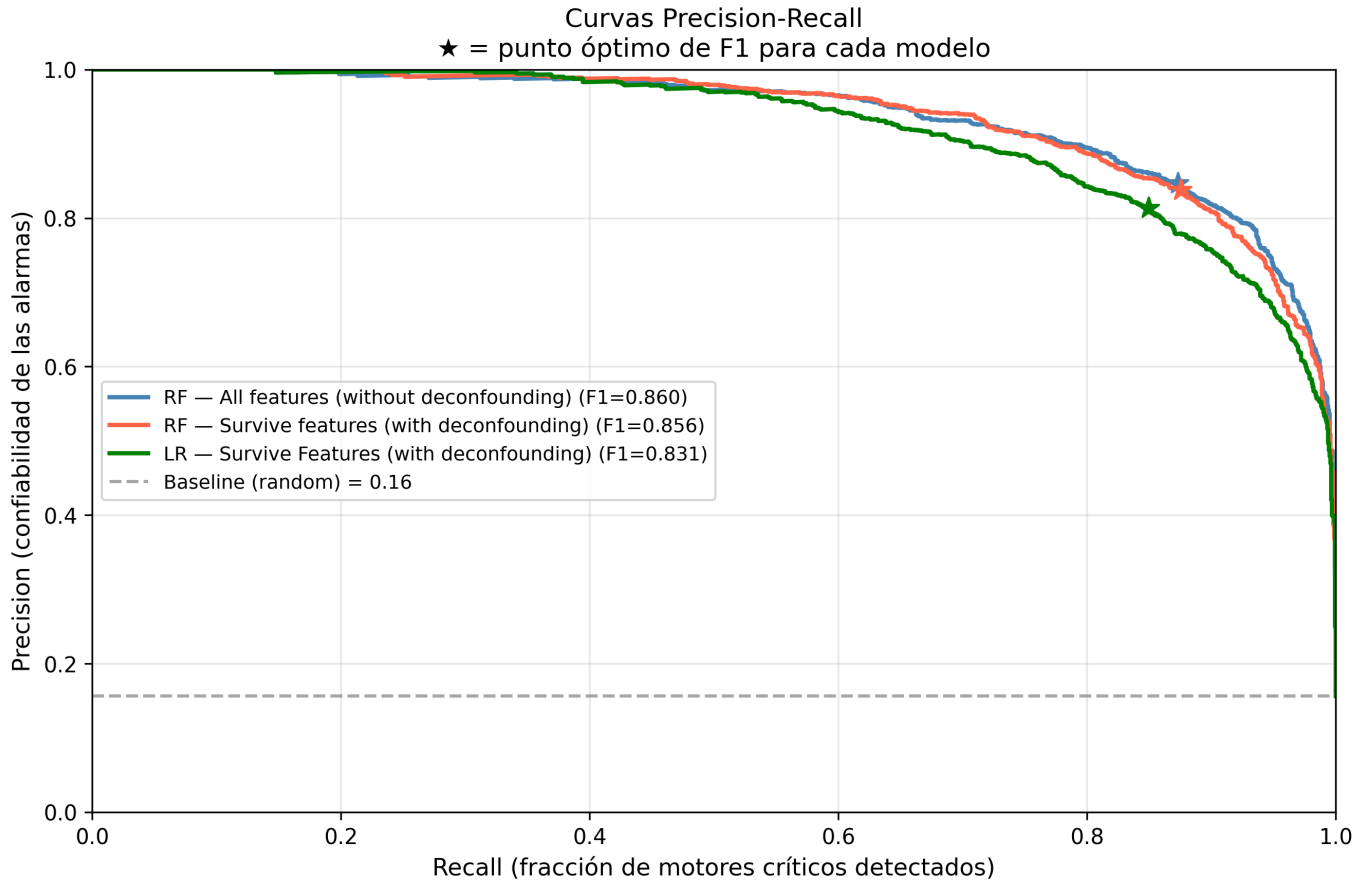


Fig. 2. Curvas Precision-Recall en el conjunto de validación. Las estrellas (★) marcan el umbral óptimo de F1 para cada modelo. La línea gris discontinua representa la línea base aleatoria (precision = 0.16).

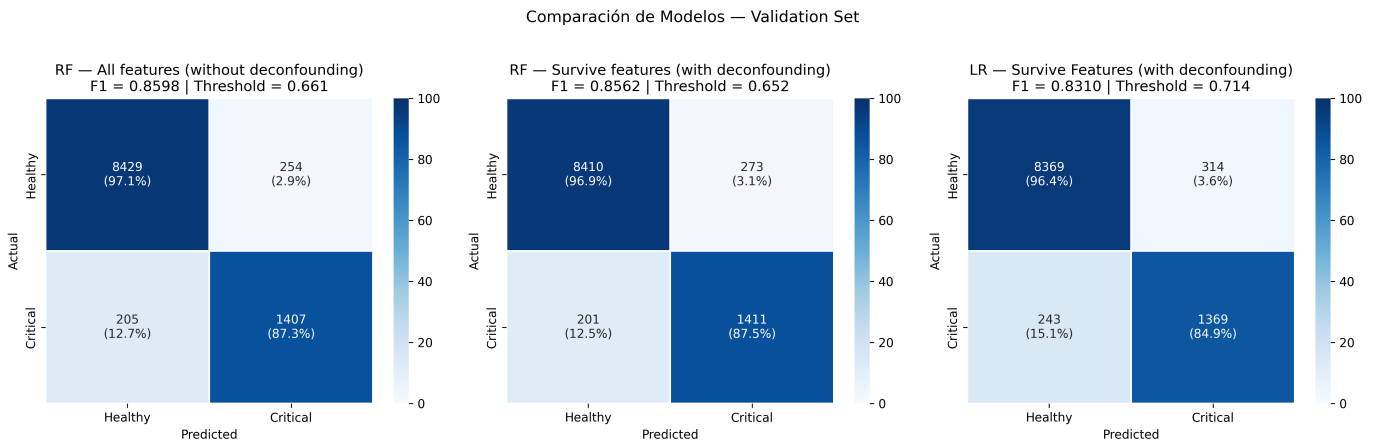


Fig. 3. Matrices de confusión sobre el conjunto de validación para los tres modelos. Los valores muestran el conteo absoluto y el porcentaje sobre la clase real.

B. Costo total esperado y comparación con escenario base

representa un costo máximo de:

Para cuantificar el beneficio económico, se compara contra el **escenario sin sistema de alerta**: todos los motores Críticos reales fallan catastróficamente. En el conjunto de validación existen 1612 ciclos Críticos reales ($N_{FN} + N_{VP}$), lo que

$$C_{\text{base}} = 1,612 \times \$500,000 = \$806,000,000 \quad (4)$$

Con el modelo activado, el costo operacional se calcula

como:

$$C_{\text{modelo}} = C_{FN} \cdot N_{FN} + C_{FP} \cdot N_{FP} + C_{VP} \cdot N_{VP} \quad (5)$$

TABLE V
COSTO OPERACIONAL Y AHORRO EN EL CONJUNTO DE VALIDACIÓN

Modelo	N_{FN}	N_{FP}	N_{VP}	C. FN	C. Mant.	Total
RF-All	205	254	1407	\$102.5M	\$24.9M	\$127.4M
RF-Sel	201	273	1411	\$100.5M	\$25.3M	\$125.8M
LR-Sel	243	314	1369	\$121.5M	\$25.3M	\$146.8M
Sin modelo	1612	0	0	\$806.0M	\$0	\$806.0M

RF-All genera un **ahorro de \$678.6M** (84.2 % del costo máximo). RF-Sel obtiene un ahorro marginalmente superior (\$680.2M) al tener 4 FN menos, aunque su F1 global sea ligeramente inferior. El costo de mantenimiento preventivo (VP + FP) representa menos del 4 % del costo total en ambos RF, confirmando que la política de alerta temprana es económicamente racional incluso con una tasa de falsas alarmas del 15 %.

C. Justificación del umbral de decisión

El umbral $\tau^* = 0.661$ fue seleccionado maximizando F1 sobre la curva Precision-Recall del conjunto de validación (Fig. 2). Bajo la estructura de costos definida, este umbral es también apropiado económicamente.

Elevar τ hacia 0.75 aumentaría la precisión a ≈ 0.91 pero reduciría el recall a ≈ 0.79 , trasladando ≈ 120 motores Críticos de VP a FN con un costo adicional de \$60M. Reducir τ a 0.50 capturaría más motores Críticos (Recall ≈ 0.95) con ≈ 400 falsas alarmas adicionales y un costo incremental de solo \$6M —tradeoff favorable bajo esta estructura asimétrica. El umbral actual representa un balance conservador que prioriza no dejar pasar fallas sobre evitar mantenimientos innecesarios, consistente con estándares de seguridad aeronáutica donde un FN tiene consecuencias irreversibles.

V. DISCUSIÓN: BIAS-VARIANCE TRADEOFF

Random Forest opera como un ensamble que reduce la varianza de árboles individuales mediante promedio de predicciones sobre distintas submuestras (bagging). La restricción $\text{max_depth}=10$ y $\text{min_samples_leaf}=20$ introduce sesgo moderado para evitar sobreajuste; cada árbol no memoriza ciclos individuales sino patrones generales de degradación.

Regresión Logística tiene mayor sesgo intrínseco al suponer una frontera lineal en el espacio de características. A pesar del escalado y la regularización L_2 ($C = 1$), no captura las interacciones no lineales entre sensores que el RF sí modela. Esto se refleja en la diferencia de F1: 0.8598 vs. 0.8310.

La desconfoundización elimina varianza espuria (mejora el sesgo de estimación causal), pero en RF-Sel reduce marginalmente el F1 respecto a RF-All, sugiriendo que el RF ya gestiona la confusión operacional implícitamente al incluir

las variables operacionales como características. En modelos lineales la desconfoundización sería más crítica para evitar que el modelo aprenda efectos de régimen en lugar de señal de degradación.

VI. CONCLUSIONES

- 1) El mejor modelo es **RF-All** (F1=0.8598), que detecta el 87.3 % de los motores críticos con especificidad del 97.1 % en la clase sana.
- 2) La desconfoundización mejora la interpretabilidad causal y permite identificar que sensores como `sensor_15` tienen correlación real de 0.53 con el fallo (vs. 0.03 sin desconfoundar), pero no produce ganancias en F1 cuando el modelo base es un Random Forest que incluye las condiciones operacionales como características.
- 3) La optimización del umbral ($\tau^* = 0.661$ vs. 0.5 por defecto) es crítica: bajo la estructura de costos definida, el umbral por defecto incrementaría el costo esperado en decenas de millones de dólares al subestimar el recall de la clase minoritaria.
- 4) **Limitaciones:** (a) No se explotan características temporales (rolling mean/std/slope) que podrían capturar la trayectoria de degradación. (b) La validación no emplea cross-validation por motor, lo que podría sobreestimar el rendimiento. (c) Se asumen costos estáticos; en la práctica dependen del tipo de falla y la flota.
- 5) **Recomendaciones:** Incorporar ventanas deslizantes de sensores, explorar Gradient Boosting (XGBoost/LightGBM) y considerar un modelo de dos etapas (regresión de RUL seguida de umbralización).

VII. CONTRIBUCIONES POR INTEGRANTE

TABLE VI
DISTRIBUCIÓN DE CONTRIBUCIONES DEL EQUIPO

Integrante	Contribución	%
Bonilla Sarmiento, Martin Jesus	Preprocesamiento, ingeniería de datos, desconfoundización, entrenamiento y evaluación de modelos, análisis de costo-beneficio, redacción del informe.	100
Watson Guevara, Natalia	Redacción del informe y evaluación de modelos.	100
Cartagena Miranda, Lucia Jimena	Redacción del informe y evaluación de modelos.	100

APPENDIX A CÓDIGO FUENTE

El código completo (con semilla `random_state=42` en todos los generadores aleatorios) está disponible en el repositorio del proyecto:

https://github.com/marbs23/ML_Project_1

El script principal `NASA_problem.py` ejecuta en orden: carga de datos, ingeniería de características, desconfoundización, entrenamiento de los tres modelos, optimización de umbral y exportación de gráficos a `outputs/`.

APPENDIX B SENSORES SUPERVIVIENTES

Tras aplicar el umbral de varianza ($\tau_v = 0.1$) y el umbral de correlación ($|\rho| \geq 0.03$ sobre sensores desconfoundizados), los **11 sensores finales** utilizados por RF-Sel y LR-Sel son:

TABLE VII
SENSORES SUPERVIVIENTES Y CORRELACIÓN CON LA ETIQUETA TRAS DESCONFOUNDIZACIÓN (VALOR ABSOLUTO)

Sensor	$ \rho $	Sensor	$ \rho $
sensor_15	0.5315	sensor_17	0.1959
sensor_11	0.3836	sensor_3	0.1940
sensor_4	0.3201	sensor_9	0.1242
sensor_13	0.2939	sensor_21	0.0528
sensor_14	0.2650	sensor_20	0.0526
sensor_2	0.0333		

APPENDIX C COMPETENCIA KAGGLE

El modelo RF-All fue enviado a la competencia *Mantenimiento Predictivo Aeronáutico* en Kaggle aplicando el umbral óptimo $\tau^* = 0.661$ sobre la última fila de cada motor en `test_FD002.txt` (259 motores). El submission obtuvo un **F1-score público de 0.90**, superando el 0.8598 obtenido en validación local, lo que confirma que el modelo generaliza correctamente a motores no vistos durante el entrenamiento.

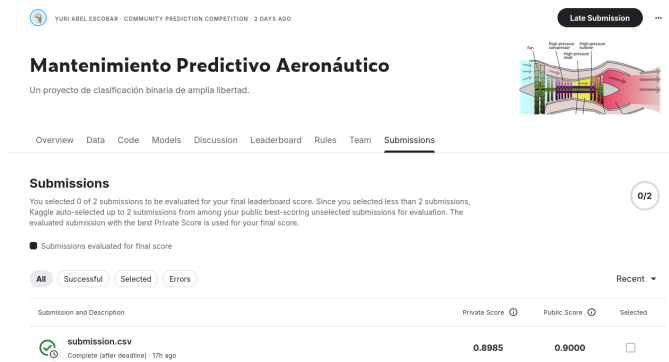


Fig. 4. Evidencia de entrega en competencia Kaggle “Mantenimiento Predictivo Aeronáutico”. F1-score público: **0.90**.